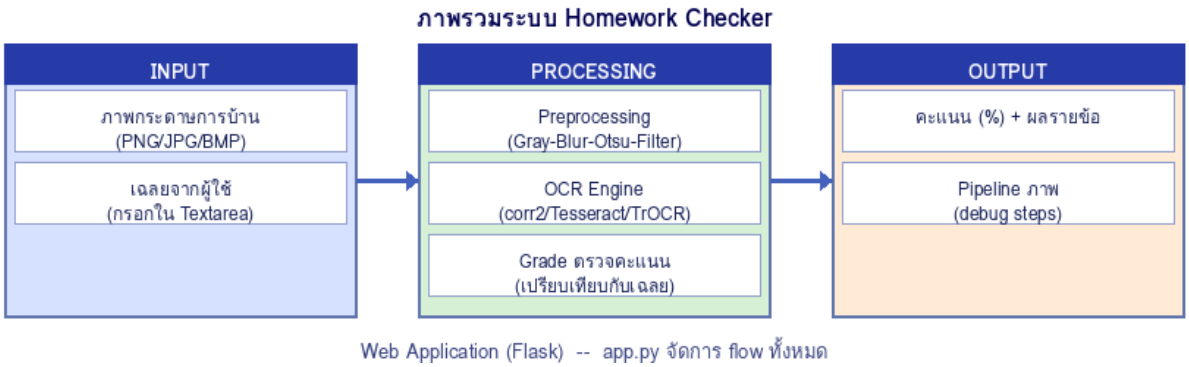
**Project Name:** Homework Checker — ตรวจการบ้านด้วย OCR

**Creator:** นายอัทธเมศร์ พิธาพัฒน์ธนกุล 2211311705

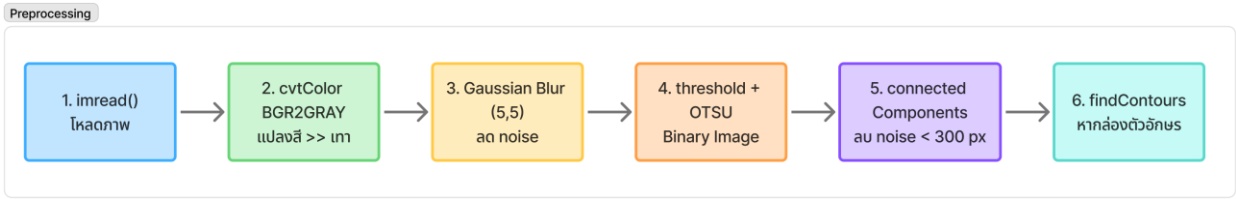
นายนวัตกรณ์ นามบุรินทร์ 2211311804

**Descriptions:** โปรแกรมตรวจการบ้านอัตโนมัติ รับภาพกระดาษการบ้านที่มีคำตอบเป็นตัวเลขหรือตัวอักษร แล้วใช้ 3 วิธีในการอ่านตัวอักษร ได้แก่ เลือก engine ได้จากหน้าเว็บ รองรับ Compare Mode (รัน 3 วิธีพร้อมกัน) และรับเฉลยจากผู้ใช้โดยตรง ใช้ 3 วิธี ได้แก่ **(1) Correlation 2D (corr2)** หลักการเดียวกับ MATLAB ของอาจารย์ **(2) Tesseract OCR** สำหรับตัวพิมพ์ภาษาอังกฤษและตัวเลข **(3) TrOCR** (Microsoft) สำหรับลายมือ ทำงานผ่านเว็บแอปพลิเคชัน Flask พัฒนาด้วย Python + OpenCV




*ภาพรวมระบบ: ภาพกระดาษการบ้าน >> Preprocessing >> OCR >> ตรวจคะแนน >> แสดงผล*

## ขั้นตอน Preprocessing




*ขั้นตอน Preprocessing ทั้ง 6 ขั้น*

**รายละเอียดแต่ละขั้น:**

1. imread() — โหลดภาพจากไฟล์เป็น BGR array (3 channels) ด้วย OpenCV

2. cvtColor (BGR2GRAY) — แปลงภาพสี 3 channels ให้เป็น Grayscale 1 channel ค่า pixel 0–255 เทียบเท่า rgb2gray

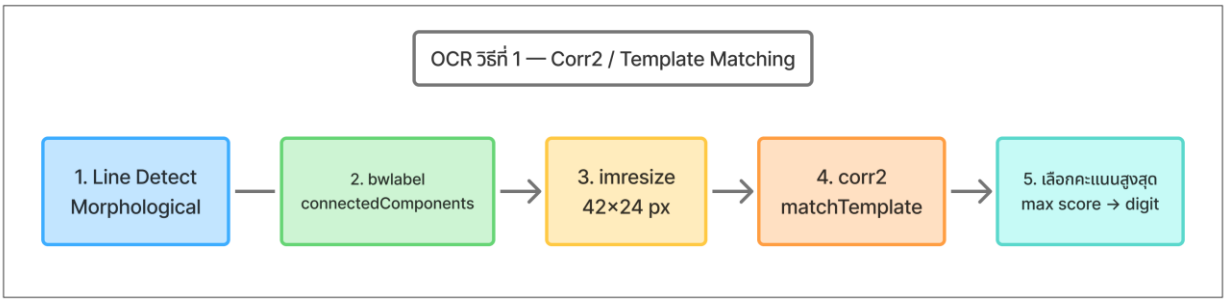
3. GaussianBlur (kernel 5×5) — เบลอภาพเล็กน้อยเพื่อกำจัด noise ระดับ pixel ก่อนทำ threshold หาก noise มีมากโดยไม่เบลอก่อน Otsu อาจเลือก threshold ผิดพลาด

4. Otsu Threshold (THRESH_BINARY_INV + THRESH_OTSU) — แปลงภาพ Grayscale เป็น Binary Image โดยอัลกอริทึม Otsu จะหาค่า threshold ที่แบ่งแยก pixel สองกลุ่ม (หมึก/กระดาษ) ได้ดีที่สุดโดยอัตโนมัติ ผลลัพธ์คือ pixel ตัวเลข = 255 (ขาว) พื้นหลัง = 0 (ดำ)

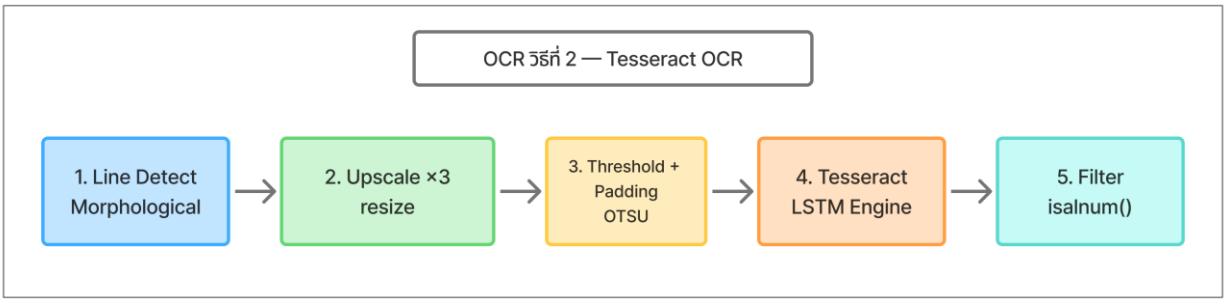
5. connectedComponentsWithStats (area ≥ 300 px) — วิเคราะห์ connected component ทั้งหมดในภาพ Binary แล้วลบกลุ่มที่มีพื้นที่น้อยกว่า 300 pixel ทิ้ง ช่วยกำจัด noise จุดเล็กที่อาจรบกวนการหากล่อง

6. Morphological Line Detection + findContours — สกัดเส้นแนวนอน (kernel 40×1) และแนวตั้ง (kernel 1×25) ด้วย MORPH_OPEN แล้วรวมเป็น box_mask จากนั้น MORPH_CLOSE (5×5, 2 รอบ) เชื่อมมุมที่ขาด กรองด้วย: พื้นที่ > 0.3% ของภาพ, ตำแหน่ง y > 5% ของความสูง (ตัด header), อัตราส่วน w/h อยู่ระหว่าง 1.0–15.0

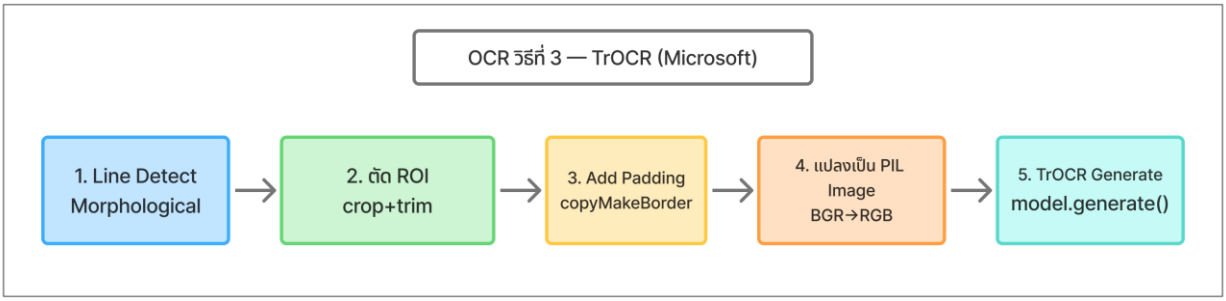
**ขั้นตอน OCR วิธีที่ 1 — Corr2 / Template Matching**




**ขั้นตอน OCR วิธีที่ 2 — Tesseract OCR**



## ขั้นตอน OCR วิธีที่ 3 — TrOCR (Microsoft)




**Conceptual:**

1. แบ่งโปรแกรมเป็น 3 ส่วนหลัก

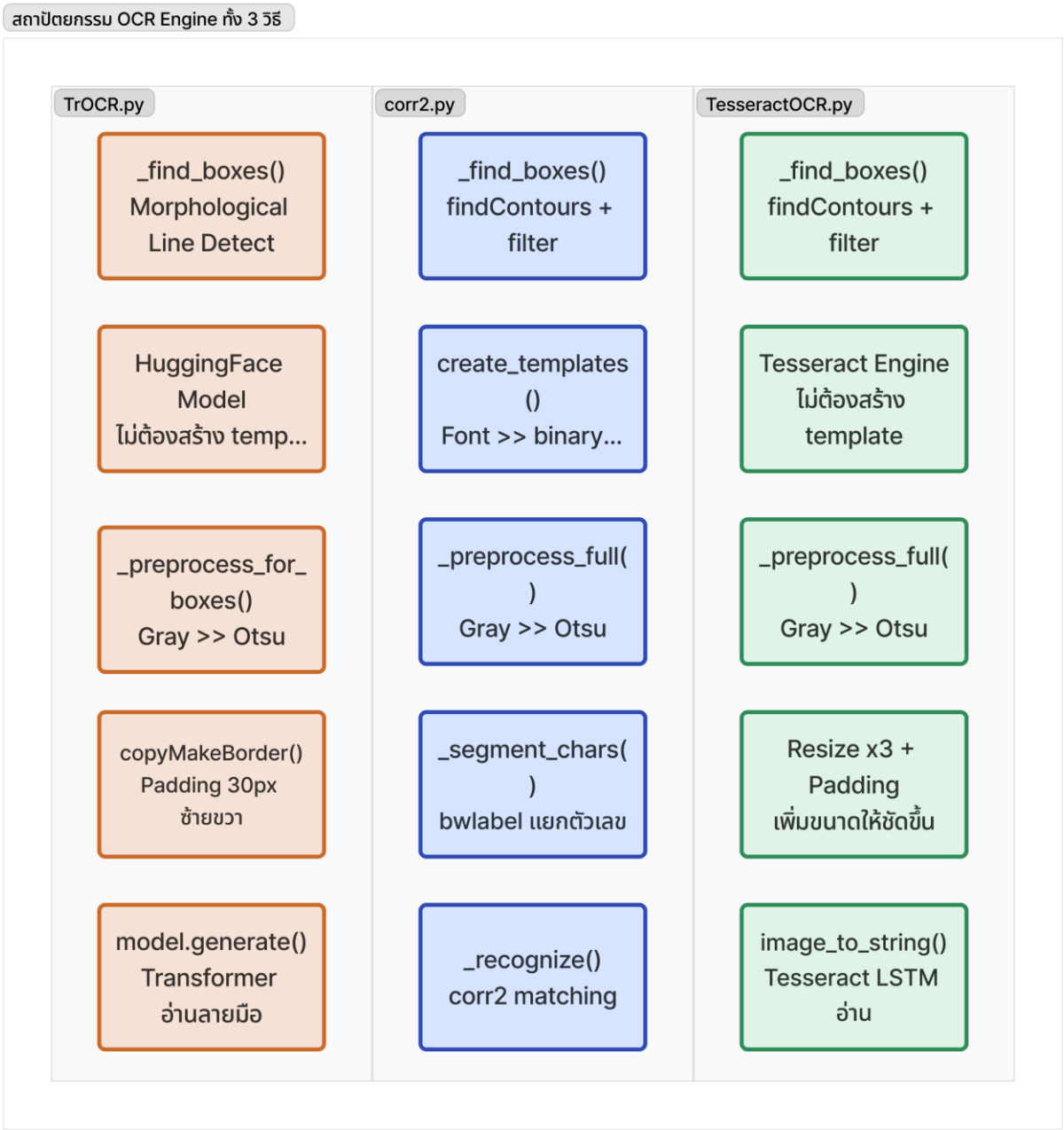
- app.py — Web Application (Flask) รับภาพและเฉลยจากผู้ใช้
- ocr_corr2.py, ocr_teserract.py, ocr_trocr.py — OCR Engine (3 ไฟล์แยกตาม engine)
- checker_corr2.py, checker_teserract.py, checker_trocr.py — Grading Logic (3 ไฟล์แยกตาม engine)

2. OCR Engine ทั้ง 3 วิธีใช้ Public API เหมือนกัน

- extract_answers(image_path) >> list[str]
- extract_answers_debug(image_path) >> (answers, pipeline_steps)

3. คำสั่งหลักที่สำคัญแต่ละวิธี

- ocr_corr2.py >> cv2.matchTemplate (TM_CCOEFF_NORMED)
- ocr_teserract.py >> pytesseract.image_to_string(roi, config='--oem 3 --psm 7')
- ocr_trocr.py >> model.generate(pixel_values, max_new_tokens=15)




*สถาปัตยกรรม OCR Engine ทั้ง 3 วิธีเปรียบเทียบกัน*

**วิธีทดสอบโปรแกรม**

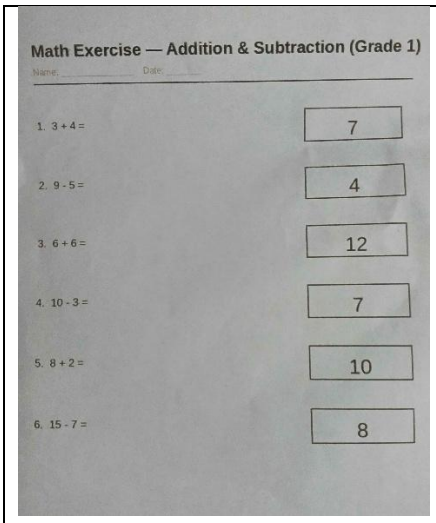
1. ติดตั้ง Dependencies

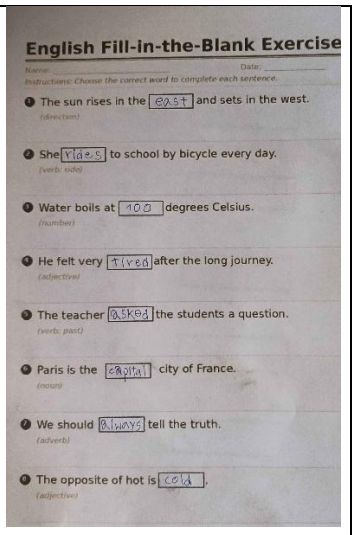
- Corr2: pip install flask opencv-python-headless pillow numpy werkzeug
- Tesseract: ติดตั้ง Tesseract-OCR Engine + pip install pytesseract
- TrOCR: pip install torch transformers

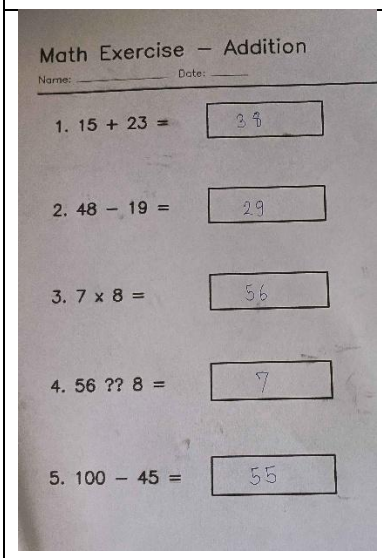
2. ไฟล์ในโปรเจค

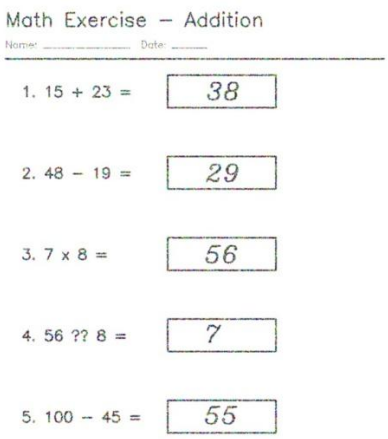
- app.py, ocr_corr2.py, ocr_teserract.py, ocr_trocr.py, checker_corr2.py, checker_teserract.py, checker_trocr.py
- templates/ — index.html, result.html, compare.html, base.html

3. ภาพตัวอย่างสำหรับทดสอบ






4. เลือก OCR Engine (จาก Dropdown ในหน้าเว็บ — ไม่ต้องแก้ import ใน app.py)

```
engine = "corr2"       # วิธี 1: Template Matching
engine = "teserract"   # วิธี 2: Tesseract OCR
engine = "trocr"       # วิธี 3: Microsoft TrOCR
```

5. รัน Web Application

```
python app.py   # เปิดเบราว์เซอร์ที่ http://localhost:5000
```

```
/* ------------------------------------------------- */
/* Code: ocr_corr2.py  — OCR วิธีที่ 1: Correlation 2D */
```

```
/* ------------------------------------------------- */
TMPL_H, TMPL_W = 42, 24

# สร้าง Template จาก Font (แทน bmp ของอาจารย์)
def create_templates():
    for fp in _FONT_CANDIDATES:
        font = ImageFont.truetype(fp, size=80)
        for d in DIGITS:
            t = _make_template(d, font)  # วาด >> crop >> resize >> threshold

# จดจำตัวเลข 1 ตัว (เหมือน read_letter.m)
def _recognize(char_img, templates):
    resized = cv2.resize(char_img, (TMPL_W, TMPL_H))  # imresize([42 24])
    digit_scores = {d: max(matchTemplate(bw, t) for t in tlist)
                    for d, tlist in templates.items()}
    best_label = max(digit_scores, key=digit_scores.get)
    return best_label, digit_scores[best_label]
/* ------------------------------------------------- */
```

/* ---------------------------------------------- */

**/* Code: ocr_teserract.py — OCR วิธีที่ 2: Tesseract */**

/* ---------------------------------------------- */

```
import pytesseract


def extract_answers(image_path):
    img             = cv2.imread(image_path)
    bw_clean, _     = _preprocess_full(img)
    boxes           = _find_boxes(bw_clean, img.shape)
    custom_config  = r'--oem 3 --psm 7'
    for (y,x,w,h) in boxes:
        roi_gray   = cv2.cvtColor(img[y:y+h,x:x+w], cv2.COLOR_BGR2GRAY)
        roi_resize = cv2.resize(roi_gray, None, fx=3, fy=3,
                                interpolation=cv2.INTER_CUBIC)
        _, roi_bin = cv2.threshold(roi_resize,0,255,
                                   cv2.THRESH_BINARY|cv2.THRESH_OTSU)
        roi_pad    = cv2.copyMakeBorder(roi_bin,20,20,20,20,
                                        cv2.BORDER_CONSTANT,value=255)
        text = pytesseract.image_to_string(roi_pad,config=custom_config)
        answers.append("".join(c for c in text if c.isalnum()))
```

/* ---------------------------------------------- */

/* ---------------------------------------------- */

**/* Code: ocr_trocr.py  — OCR วิธีที่ 3: Microsoft TrOCR */**

/* ---------------------------------------------- */

```
from transformers import TrOCRProcessor, VisionEncoderDecoderModel


processor = TrOCRProcessor.from_pretrained("microsoft/trocr-base-handwritten")
model     = VisionEncoderDecoderModel.from_pretrained(...).to(device)


# หากล่องด้วย Morphological Line Detection
def _find_boxes(bw, img_shape):
    hor = cv2.morphologyEx(bw, cv2.MORPH_OPEN, kernel_30x1)
    ver = cv2.morphologyEx(bw, cv2.MORPH_OPEN, kernel_1x20)
    box_mask = cv2.add(hor, ver)
    ...


# อ่านด้วย Vision Transformer
def _recognize_trocr(roi):
    roi_pad = cv2.copyMakeBorder(roi,30,30,30,30,value=[255,255,255])
    pil_img = Image.fromarray(cv2.cvtColor(roi_pad,cv2.COLOR_BGR2RGB))
    px      = processor(images=pil_img,return_tensors="pt").pixel_values
    ids     = model.generate(px.to(device), max_new_tokens=15)
    return processor.batch_decode(ids,skip_special_tokens=True)[0].strip()
```

/* ---------------------------------------------- */

## ผลการทดสอบ

### ใช้วิธี Corr2 (Template Matching)

รองรับเฉพาะตัวเลข เนื่องจากมีความคล้ายกันของ Font Template จึงไม่นำตัวอักษรมาทดสอบ

*ตัวเลขตัวพิมพ์*

*ตัวเลขลายมือ*

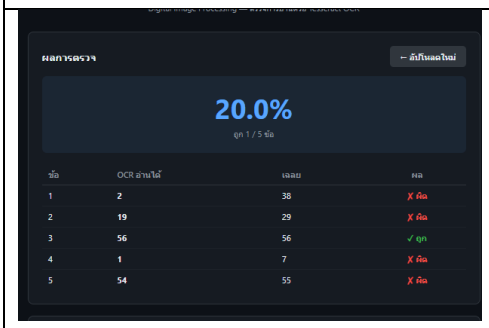
**ผลพบว่า:** ตัวเลขตัวพิมพ์ใช้ได้ดี แต่ลายมือยังใช้ไม่ได้ เนื่องจาก template สร้างจาก font ที่ต่างจากลายมือ

### ใช้ Tesseract OCR

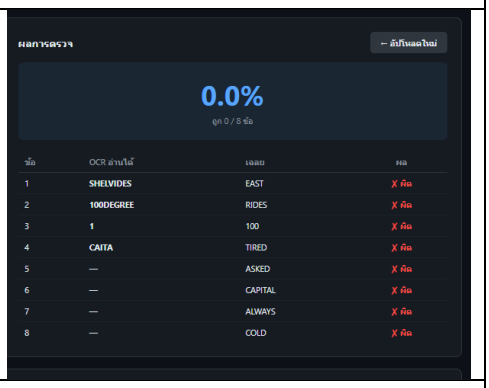
ทดสอบกับทั้งตัวเลขและตัวอักษรภาษาอังกฤษ ทั้งแบบตัวพิมพ์และลายมือ

ลายมือ

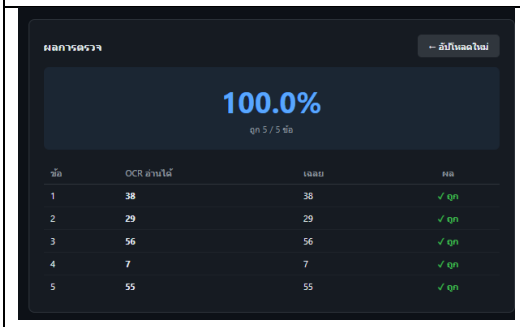
ตัวเลข

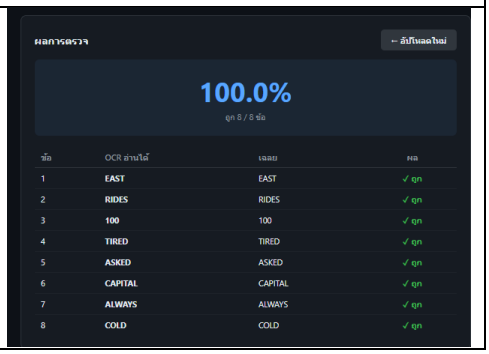



ตัวอักษร




ตัวพิมพ์



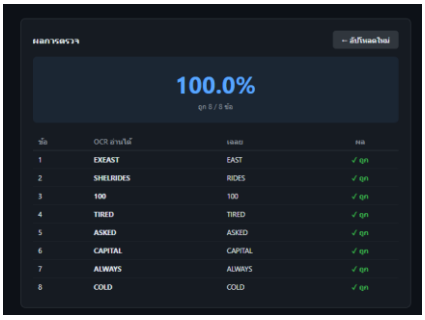




**ผลพบว่า:** ตัวเลขและอักษรตัวพิมพ์ทำงานได้ดี แต่ลายมือยังจับไม่ได้ เนื่องจาก Tesseract ถูกฝึกกับตัวพิมพ์เป็นหลัก

## ใช้ TrOCR — Microsoft

ทดสอบกับลายมือเป็นหลัก เนื่องจากโมเดลถูก pre-train ด้วย handwritten dataset

*ตัวเลขลายมือ*




*ตัวอักษรลายมือ*

**ผลพบว่า:** TrOCR ทำงานได้ดีกับตัวอักษรลายมือ แต่ตัวเลขยังใช้ไม่ได้

**ตารางสรุปผลการทดสอบ**

| คุณสมบัติ | Corr2 (Template) | Tesseract OCR | TrOCR (Microsoft) |
|---|---|---|---|
| หลักการ | Correlation 2D Template Matching | LSTM + CTC (Rule-based) | Transformer (Vision Encoder) |
| ประเภทที่รองรับ | ตัวเลข 0-9 (ตัวพิมพ์) | ตัวเลข + อักษร EN (ตัวพิมพ์) | ตัวเลข + อักษร EN (ลายมือ) |
| ตัวเลข (ตัวพิมพ์) | ดีมาก | ดีมาก | ดี |
| ตัวเลข (ลายมือ) | ไม่ดี | ไม่ดี | ไม่ดี |
| อักษร EN (ตัวพิมพ์) | ไม่รองรับ | ดีมาก | ดีมาก |
| อักษร EN (ลายมือ) | ไม่รองรับ | ไม่ดี | ดี |
| ความเร็ว | เร็วมาก (CPU) | เร็ว (CPU) | ช้า (ต้องการ GPU) |

*สรุปผลการทดสอบความแม่นยำของทั้ง 3 วิธีแยกตามประเภทข้อมูล*

**คำสั่งที่เรียนรู้และใช้งาน**

| คำสั่ง / ฟังก์ชัน | คำอธิบาย |
|---|---|
| cv2.imread(filename) | โหลดภาพจากไฟล์ — เทียบเท่า imread ใน MATLAB |
| cv2.cvtColor(BGR2GRAY) | แปลงเป็น Grayscale — เทียบเท่า rgb2gray |
| cv2.GaussianBlur(img,(5,5),0) | Gaussian blur ลด noise ก่อน threshold |
| cv2.threshold(THRESH_OTSU) | Otsu threshold อัตโนมัติ — เทียบเท่า graythresh+im2bw |
| cv2.connectedComponentsWithStats() | หา connected components — เทียบเท่า bwlabel |
| cv2.findContours(...) | ค้นหา contour จาก box_mask เพื่อหากล่องคำตอบ (ใช้ร่วมกับ Morphological Line Detection) |
| cv2.morphologyEx(MORPH_OPEN) | สกัดเส้นแนวนอน/ตั้ง — ใช้ใน Morphological Line Detection ทั้ง 3 engine |
| cv2.morphologyEx(MORPH_CLOSE) | เชื่อมมุมกล่องให้สนิท |
| cv2.resize(img,(W,H)) | Resize ภาพ — เทียบเท่า imresize([42 24]) |
| cv2.copyMakeBorder(...) | เพิ่ม padding สีขาวรอบ ROI ก่อนส่ง OCR |
| cv2.matchTemplate(TM_CCOEFF_NORMED) | Normalized cross-correlation — corr2 |
| np.mean(A) | ค่าเฉลี่ย — ใช้คำนวณ InputAVG, PatternAVG ใน corr2 |
| np.sum((A-Am)*(B-Bm)) | Numerator ของสูตร corr2 |
| ImageFont.truetype(path,size) | โหลด TrueType font สร้าง digit template |
| pytesseract.image_to_string() | อ่านข้อความด้วย Tesseract LSTM Engine |
| --oem 3 --psm 7 | Tesseract: LSTM engine + single line mode |
| TrOCRProcessor.from_pretrained() | โหลด preprocessor สำหรับ TrOCR |
| VisionEncoderDecoderModel() | โหลด Vision Transformer model ของ Microsoft |

| | |
|---|---|
| model.generate(pixel_values) | ให้ Transformer อ่านข้อความจากภาพ |
| processor.batch_decode(ids) | แปลง token ids กลับเป็นข้อความ |
| render_template(name,**ctx) | แสดง HTML ผลลัพธ์ผ่าน Flask |
| corr2(A,B) | คำนวณ 2D Correlation — หัวใจของวิธีที่ 1 |
| create_templates() | สร้าง template 0–9 — เทียบเท่า create_templates.m |
| extract_answers(path) | Public API: อ่านคำตอบทุกกล่อง (ทั้ง 3 วิธี) |
| grade(image_path, provided_key) | ตรวจการบ้าน: เปรียบเทียบกับเฉลย คืนคะแนน — รองรับ provided_key (กรอกจากหน้าเว็บ) และ answer_key.json |